

Mental Buddy – CLAUDE.md

Vollständige App-Spezifikation für die KI-gestützte Entwicklung

Was ist Mental Buddy?

Mental Buddy ist eine **kostenlose, spielerische Lernplattform für mentale Gesundheit** – entwickelt für Menschen die sich zu spät oder gar nicht Hilfe holen, besonders Männer. Die App ist kein Therapieersatz, sondern ein täglicher Begleiter der wissenschaftlich fundierte Methoden durch Wiederholung und Spielmechanik in den Alltag bringt.

Plattform: iOS & Android (React Native) **Kosten:** 100% kostenlos, keine Werbung, kein Abo
Sprache: Deutsch (später mehrsprachig) **Wissenschaftliche Basis:** CBT, ACT, DBT, MBSR, MSC – transparent benannt, verständlich erklärt

Technischer Stack

```
Frontend:    React Native (Expo)
Navigation:  React Navigation (Bottom Tabs + Stack)
State:       Zustand
Storage:     AsyncStorage (lokal, kein Cloud-Zwang)
Audio:       Expo AV
Animation:   React Native Reanimated + Lottie
Charts:      Victory Native
DB lokal:    SQLite via expo-sqlite
Backend:     Node.js + Express (optional, nur für Community-Feature)
```

Projektstruktur

```
mental-buddy/
├── app/
│   ├── screens/
│   │   ├── OnboardingScreen.tsx
│   │   ├── HomeScreen.tsx
│   │   └── LibraryScreen.tsx      # Verstehen
```

		PracticeScreen.tsx	# Üben
		CharacterScreen.tsx	# Charakter & Fortschritt
		PlannerScreen.tsx	# Wochenplan & Ziele
		ReflectScreen.tsx	# Reflektieren
		HelpScreen.tsx	# Hilfe finden
		AcuteHelpScreen.tsx	# NEU: Akut-Hilfe Flow
		CommunityScreen.tsx	# Stille Solidarität
		components/	
		SkillCard.tsx	
		ExercisePlayer.tsx	# Timer + Audio + Animation
		CharacterAvatar.tsx	
		GoalTracker.tsx	
		WeeklyPlanner.tsx	
		EnergyBar.tsx	
		ProgressReport.tsx	
		TherapyHint.tsx	# Immer abrufbarer Hinweis
		HelpNowButton.tsx	# NEU: Globaler Akut-Button
		ModeToggle.tsx	# NEU: Klartext-Modus Schalter
		data/	
		skills.ts	# Alle Skills & Übungen
		topics.ts	# Themen-Bibliothek
		enemies.ts	# Gegner-Definitionen (nur RPG-Modus)
		equipment.ts	# Ausrüstung & Belohnungen
		acuteStates.ts	# NEU: Zustände für Akut-Hilfe
		labels.ts	# NEU: Doppelte Labels (RPG/Klartext)
		store/	
		userStore.ts	# Nutzerprofil & Charakter
		progressStore.ts	# XP, Level, Skills
		plannerStore.ts	# Ziele & Wochenplan
		reflectionStore.ts	# Tagebuch & Notizen
		settingsStore.ts	# NEU: Modus (RPG/Klartext) + Einstellungen
		utils/	
		spacedRepetition.ts	# SM-2 Algorithmus
		reportGenerator.ts	# PDF-Export
		therapyMatcher.ts	# Hilfe-Orientierung
		acuteMatcher.ts	# NEU: Zustand → passende Übung
		assets/	
		animations/	# Lottie JSON
		audio/	# Geführte Übungen (MP3)
		character/	# Charakter-Sprites & Equipment
		CLAUDE.md	

Die sechs Säulen – Implementierung

SÄULE 1 – Verstehen (`LibraryScreen`)

Themen-Bibliothek die der Nutzer frei durchsuchen kann.

Themen (vollständige Liste):

- Angst & Panik
- Antriebslosigkeit & Depression
- Innerer Kritiker & Selbstzweifel
- Wut & Frustration
- Trauer & Verlust verarbeiten
- Einsamkeit & soziale Isolation
- Wenn alles zu viel wird (Overwhelm)
- Stress & Burnout
- Schlafprobleme
- Prokrastination & Lähmung
- Trauma bewältigen
- Umgang mit mentaler Gesundheit in der Beziehung
- Tut mir meine Beziehung noch gut?
- Selbstwert & Identität

Pro Thema:

```
interface Topic {  
    id: string  
    title: string           // Klare direkte Bezeichnung  
    introText: string       // Sanfte Einführung (Onboarding-Phase)  
    deepText: string        // Klare direkte Erklärung (nach Woche 2)  
    relatedSkills: string[]  // Verknüpfte Skills  
    relatedEnemies: string[] // Welche Gegner dieses Thema erzeugt (nur RPG-Modus)  
    therapyNote: string     // Wann professionelle Hilfe wichtig ist  
    isSensitive: boolean    // Trigger für verstärkten Therapie-Hinweis  
}
```

Sprachprogression:

- Erste 2 Wochen: `introText` – metaphorisch, sanft, einladend

- Ab Woche 3: `deepText` – klare Begriffe (Depression, Trauma, Angststörung)
 - `isSensitive: true` → immer `TherapyHint` einblenden
-

SÄULE 2 – Üben (`PracticeScreen` + `ExercisePlayer`)

Der Kern der App. Jede Übung wird aktiv begleitet – nicht nur beschrieben.

Übungs-Typen:

```
type ExerciseType = 'text' | 'timer' | 'animation' | 'audio' | 'combined'

interface Exercise {
  id: string
  skillId: string
  level: 1 | 2 | 3 | 4 | 5
  type: ExerciseType
  title: string
  durationSeconds: number
  instructions: string[]      // Schritt-für-Schritt
  audioFile?: string         // Expo AV
  animationFile?: string     // Lottie
  timerPattern?: number[]    // z.B. [4, 7, 8] für 4-7-8 Atem
  xpReward: number
  reflectionPrompt: string   // Frage nach der Übung
  isAcute?: boolean          // NEU: Eignet sich für Akut-Hilfe
  acuteStates?: string[]     // NEU: Passende Akut-Zustände
}
```

`ExercisePlayer` Komponente:

- Zeigt Schritt-für-Schritt-Anleitung
 - Spielt Audio ab (Expo AV)
 - Zeigt Lottie-Animation (Atemkreis, Körper-Scan etc.)
 - Timer mit visueller Kreisanzeige
 - Abschluss → XP-Animation → Reflexionsfrage → Charakter-Update
-

SÄULE 3 – Wiederholen (Spaced Repetition)

Algorithmus: SM-2 (vereinfacht)

```
// utils/spacedRepetition.ts
interface SkillReview {
  skillId: string
  lastSeen: Date
  interval: number // Tage bis zur nächsten Wiederholung
  easeFactor: number // 1.3 - 2.5
  repetitions: number
}

function getNextReview(review: SkillReview, quality: 0|1|2|3|4|5): SkillReview {
  // quality: 0-2 = nicht beherrscht → heute wiederholen
  // quality: 3-5 = beherrscht → Intervall verlängern
  // Neue Skills: täglich
  // Bekannte Skills: 3 Tage → 1 Woche → 2 Wochen → 1 Monat
}
```

Vergangene Erfolge als neue Level:

- Skills die gemeistert wurden kommen als Level+1 zurück
- Framing: *"Du erinnerst dich? Damals war das schwer. Jetzt schaffst du Level 2."*
- Erzeugt Erfolgserlebnisse auch bei Rückschritten

SÄULE 4 – Reflektieren (ReflectScreen)

Selbstreflexion nach Übungen:

```
interface Reflection {
  id: string
  exerciseId: string
  date: Date
  prompt: string
  answer: string
  mood: 1 | 2 | 3 | 4 | 5
  helpfulRating: 1 | 2 | 3 | 4 | 5
}
```

Nachricht ans zukünftige Ich:

```
interface FutureMessage {
  id: string
  content: string
  writtenAt: Date
}
```

```

    deliverAt: Date    // 1 Monat / 3 Monate / 1 Jahr
    delivered: boolean
    // Push-Notification wenn deliverAt erreicht
}

```

Persönlicher Fortschrittsbericht (PDF-Export):

Abschnitte:

1. Mein großes Ziel
 2. Skills die mir geholfen haben (mit Methoden-Namen)
 3. Aktuelle Themen für die Therapie
 4. Diese Woche (Aktivität + Fortschritt)
 5. Notizen & Reflexionen
-

SÄULE 5 – Struktur (`PlannerScreen`)

Drei-Ebenen-Zielsystem:

```

interface BigGoal {
    id: string
    title: string
    createdAt: Date
    // Kein Zeitdruck, immer sichtbar
}

interface MonthlyGoal {
    id: string
    bigGoalId: string
    title: string
    dueDate: Date
    steps: GoalStep[]
    // Verschieben erlaubt - App fragt nach, urteilt nie
}

interface GoalStep {
    id: string
    text: string
    done: boolean
    dueDate?: Date
    // Je näher Deadline → häufiger sichtbar
    // Nie als Strafe - immer als "Du wolltest das, ich bin hier"
}

```

```
}
```

Wochenplan:

```
interface WeeklyTask {  
    id: string  
    day: 'Mo' | 'Di' | 'Mi' | 'Do' | 'Fr' | 'Sa' | 'So'  
    title: string  
    color: string  
    linkedExerciseId?: string // Optional: direkt mit Übung verknüpft  
    done: boolean  
    reminderTime?: string     // Optional, sanft  
}
```

Kernprinzip: Nutzer entscheidet selbst ob er bereit für mehr ist. Zurückgehen ist kein Versagen.

SÄULE 6 – Hilfe finden (`HelpScreen`)

Orientierungshilfe (keine Diagnose):

5 Fragen die helfen zu verstehen welche Art Unterstützung sinnvoll wäre:

1. Wie lange belastet dich das schon? (Tage / Wochen / Monate / Jahre)
2. Wie stark beeinträchtigt es deinen Alltag? (Skala 1–5)
3. Hast du schon mal mit jemandem darüber gesprochen?
4. Was erhoffst du dir von Unterstützung?
5. Hast du Bedenken wegen Kosten oder Wartezeiten?

→ Ergebnis: Klare Empfehlung welche Art Hilfe passen könnte (Psychotherapeut, Psychiater, Beratungsstelle, Coaching, Krisentelefon)

Informations-Seiten:

- Was ist der Unterschied zwischen Psychologe / Psychiater / Psychotherapeut?
- Wie finde ich einen Kassenplatz? (Schritt-für-Schritt)
- Was kostet Therapie? Was zahlt die Kasse?
- Wie läuft eine erste Stunde ab?
- Notfall: Krisentelefon **0800 111 0 111** (immer sichtbar)

Therapiebegleitung (für Nutzer in Therapie):

- Themen-Notizen jederzeit erfassen
 - Vor-der-Stunde Zusammenfassung
 - Nach-der-Stunde Reflexion
 - Export als PDF für Therapeuten (nur auf Wunsch)
-

Hilfe-Button (Akut-Übungen)

Prinzip: Es geht nicht darum, Krisen zu erkennen – sondern darum, im Moment konkret zu helfen. Wer jetzt Unterstützung braucht, bekommt sofort eine passende Übung. Kein Menü, kein Scrollen, keine Erklärungen.

Globaler Button: Immer sichtbar als `HelpNowButton` im Header jedes Screens (außer während einer laufenden Übung). Klar beschriftet: **“Jetzt helfen”**.

Abgrenzung zum Krisentelefon: Das Krisentelefon bleibt im `HelpScreen` immer sichtbar. Der Akut-Button führt zu *Übungen*, nicht zu Hotlines. Beides hat seinen Platz, beides ist sofort erreichbar.

Flow (`AcuteHelpScreen`):

1. **Eine Frage:** *“Was brauchst du gerade?”*
2. **4–6 konkrete Zustände** als große Tap-Flächen (keine Skalen, keine Erklärungen)
3. **Direkt zur passenden Übung** (2–5 Minuten)
4. Nach der Übung: *“Hat das geholfen?”* → optional gespeichert für bessere Vorschläge beim nächsten Mal

Zustände & passende Übungen:

```
// data/acuteStates.ts
interface AcuteState {
  id: string
  label: string // Klare Sprache, keine Metaphern
  labelPlain?: string // Alternative für Klartext-Modus (falls nötig)
  exerciseIds: string[] // Rotierend oder passend gewählt
  method: string // Transparent: CBT / DBT / MBSR / etc.
}

const ACUTE_STATES: AcuteState[] = [
```



```

{
  id: 'overwhelmed',
  label: 'Alles ist zu viel',
  exerciseIds: ['5-4-3-2-1-grounding', 'box-breathing'],
  method: 'DBT-Skills / Grounding'
},
{
  id: 'panic',
  label: 'Ich komme nicht runter',
  exerciseIds: ['4-7-8-breathing', 'cold-water-reset'],
  method: 'Atem-Regulation / DBT'
},
{
  id: 'anger',
  label: 'Ich könnte explodieren',
  exerciseIds: ['tipp-skill', 'intense-exercise-release'],
  method: 'DBT TIPP'
},
{
  id: 'spiral',
  label: 'Ich drehe mich im Kopf',
  exerciseIds: ['thought-distancing', 'name-the-thought'],
  method: 'CBT / ACT Defusion'
},
{
  id: 'empty',
  label: 'Ich fühle nichts / bin wie gelähmt',
  exerciseIds: ['body-scan-short', 'one-small-action'],
  method: 'MBSR / Aktivierung'
},
{
  id: 'self-critical',
  label: 'Ich bin hart mit mir',
  exerciseIds: ['self-compassion-break', 'kind-sentence'],
  method: 'MSC (Self-Compassion)'
}
]

```

Design-Prinzipien des Akut-Flows:

- **Keine Diagnose.** Keine Frage "Hast du Panik?" – nur "Was brauchst du?"
- **Keine Bewertung.** Wer den Button nutzt, hat richtig gehandelt.
- **Keine Gamification.** Im Akut-Flow gibt es keine XP-Animation, keine Level-Ups, keine Belohnung. Der Erfolg ist, dass die Übung gemacht wurde.

- **Schnell raus.** Nach der Übung: simpler Abschluss, Option *“Nochmal”* oder *“Danke, passt”*.
- **Methodentransparenz bleibt.** Am Ende der Übung sichtbar: *“Das war eine DBT-Technik. Mehr dazu unter [Skill].”* – damit der Nutzer später gezielt weiterüben kann.

Matching-Logik (`acuteMatcher.ts`):

```
function getAcuteExercise(stateId: string, history: AcuteUsage[]): Exercise {
  // 1. Passende Übung zum Zustand
  // 2. Bevorzuge Übungen die der Nutzer schon mal hilfreich fand
  // 3. Rotiere wenn mehrere gleich passen (nicht immer dieselbe)
  // 4. Fallback: kürzeste Übung des Zustands
}
```

Datenmodell (optional gespeichert, lokal):

```
interface AcuteUsage {
  stateId: string
  exerciseId: string
  usedAt: Date
  wasHelpful?: boolean // Nach der Übung abgefragt, optional
}
```

Über Zeit entsteht so ein persönliches Bild: *Welche Übungen helfen diesem Nutzer bei welchem Zustand?* Das verbessert künftige Vorschläge – komplett lokal, ohne Tracking.

Klartext-Modus

Prinzip: Das RPG-Framing (Charakter, Klassen, Gegner, Equipment) senkt für viele die Hürde – kann aber für andere befremdlich oder unpassend wirken, besonders wenn es gerade wirklich schlecht geht. Der Klartext-Modus bietet dieselben Inhalte ohne Spiel-Metaphern.

Einstellung: Zwei Modi, jederzeit umschaltbar in den Einstellungen. Nutzer werden am Ende des Onboardings gefragt – mit klarer Beschreibung was jeder Modus bedeutet.

```
// store/settingsStore.ts
interface Settings {
  mode: 'rpg' | 'plain' // Klartext = 'plain'
  language: 'de' | 'en'
  reminders: boolean
  reducedMotion: boolean
}
```

}

Was sich im Klartext-Modus ändert:

Element	RPG-Modus	Klartext-Modus
Charakter	"Der Denker" mit Avatar	Name + Profilbild (optional)
Level	"Level 7" mit Balken	"Woche 7 – 23 Übungen gemacht"
XP	" +20 XP" Animation	<i>entfällt</i> (nur Häkchen "Gemacht")
Skills	"Gedanken beobachten – Level 3"	"CBT: Gedanken beobachten – 3x geübt"
Gegner	"Du hast Den Nebel geschwächt"	<i>entfällt komplett</i>
Equipment	Freigeschaltete Items	<i>entfällt</i>
Fortschritt	Charakter-Aussehen ändert sich	Zeitleiste: <i>Das hast du geübt</i>
Sprache	Kumpelhaft mit Spiel-Ton	Direkt, sachlich, respektvoll

Was identisch bleibt:

- Alle Übungen, alle Methoden, alle Inhalte
- Themen-Bibliothek, Reflexionen, Wochenplan, Ziele
- Akut-Hilfe-Button
- Therapie-Hinweise und Hilfe-Finder
- Community (stille Solidarität)
- Datenschutz-Prinzipien

Technische Umsetzung:

```
// data/labels.ts - doppelte Labels
interface DualLabel {
  rpg: string
  plain: string
}
```

```
const LABELS: Record<string, DualLabel> = {
  levelUp:    { rpg: 'Level aufgestiegen!',      plain: 'Neue Übung freigeschalt' },
  xpGain:     { rpg: '+20 XP',                  plain: 'Gemacht ✓' },
}
```

```

    skillMastered: { rpg: 'Skill gemeistert',          plain: 'Du kennst diese Übung g
    dailyQuest: { rpg: 'Deine Tages-Quest',          plain: 'Übung für heute' },
    characterGrows: { rpg: 'Dein Charakter wächst', plain: 'Dein Fortschritt' },
  }

// Hook für Komponenten
function useLabel(key: string): string {
  const mode = useSettingsStore(s => s.mode)
  return LABELS[key][mode]
}

```

Wrapper-Komponenten:

- `<CharacterAvatar />` rendert im Klartext-Modus ein neutrales Profil statt RPG-Sprite
- `<EnergyBar />` wird im Klartext-Modus zu einer schlichten Fortschrittsanzeige
- `<XPAnimation />` ist im Klartext-Modus ein stummes Häkchen
- Gegner-Referenzen (`enemies.ts`) werden im Klartext-Modus **nie** angezeigt

Wechsel zwischen Modi:

- Jederzeit in Einstellungen möglich
- Kein Datenverlust: XP und Fortschritt bleiben im Hintergrund erhalten
- Wer von RPG zu Klartext wechselt, sieht sofort die sachliche Darstellung
- Wer zurückwechselt, findet seinen Charakter wie zuvor

Frage am Ende des Onboardings (Screen 8b – neu):

„Wie möchtest du die App nutzen?“

Mit Charakter & Spiel-Elementen – ein Avatar wächst mit, Übungen geben Belohnungen, das Lernen fühlt sich wie ein Spiel an.

Klartext – keine Spiel-Metaphern, nur du und die Übungen. Direkt und sachlich.

Du kannst das jederzeit in den Einstellungen ändern.“



RPG-System (nur im RPG-Modus aktiv)

Charakter

```

interface Character {
  name: string
  class: 'Denker' | 'Kämpfer' | 'Stille' | 'Sucher'
  level: number
  xp: number
  equipment: Equipment[] // Freigeschaltet durch Skills
  abilities: Ability[] // Freigeschaltet durch Übungs-Level
  appearance: AppearanceConfig // Wächst sichtbar mit Level
}

```

Charakter-Klassen:

Klasse	Stärke	Startskill
Der Denker	Reflexion & Analyse	Gedanken beobachten (CBT)
Der Kämpfer	Durchhalten & Handeln	Grounding (DBT)
Der Stille	Ruhe & Achtsamkeit	Innere Ruhe (MBSR)
Der Sucher	Verstehen & Wachsen	Innerer Anker (ACT)

Hinweis: Klasse ist nach dem Onboarding wechselbar. Menschen verändern sich.

Skills & Gegner

```

const SKILLS = [
  { id: 'thought-observation', name: 'Gedanken beobachten', method: 'CBT',
  { id: 'inner-calm',          name: 'Innere Ruhe',          method: 'MBSR',
  { id: 'grounding',          name: 'Grounding',          method: 'DBT',
  { id: 'inner-anchor',       name: 'Innerer Anker',       method: 'ACT',
  { id: 'conflict-strength',  name: 'Konflikt-Staerke',   method: 'DBT',
  { id: 'grief-work',         name: 'Verlust tragen',     method: 'Trauerarbeit
  { id: 'self-compassion',    name: 'Selbstmitgefuehl',   method: 'MSC',
]

const ENEMIES = [
  { id: 'fog',    name: 'Der Nebel',    description: 'Antriebslosigkeit',    wea
  { id: 'noise',  name: 'Der Laerm',    description: 'Gedankenspirale/Angst', wea
  { id: 'freeze', name: 'Die Starre',    description: 'Prokrastination',      wea
  { id: 'shadow', name: 'Der Schatten', description: 'Innerer Kritiker',      wea
]

```

Wichtig: Gegner werden **nie im Akut-Flow** gezeigt – auch nicht im RPG-Modus. Wer

gerade leidet, soll keinen "Gegner besiegen" müssen.

XP & Equipment

- Übung abschließen → XP je nach Level der Übung (10–50 XP)
 - Skill auf Level X bringen → Ausrüstungsstück freischalten
 - Gegner besiegen → besondere Belohnungen
 - Charakter-Aussehen ändert sich sichtbar alle 5 Level
-

Onboarding-Flow

Screen 1: Begrüßung

- "Hallo. Hier wird nichts bewertet."
- Kein Login-Zwang (optional für Backup)

Screen 2: Charakter / Profil erstellen

- Name eingeben
- Klasse wählen (Denker / Kämpfer / Stille / Sucher)
- Kurze Beschreibung was jede Klasse bedeutet
- (Im Klartext-Modus später: nur Name)

Screen 3-7: 5 kurze Fragen (eine pro Screen)

1. "Was raubt dir gerade am meisten Energie?" (Mehrfachauswahl aus Themen)
2. "Wie lange ist das schon so?" (Schieberegler: Tage → Monate → Jahre)
3. "Nutzt du die App alleine oder begleitend zur Therapie?"
4. "Was wäre für dich ein echter Erfolg in 3 Monaten?" (Freitext)
5. "Wie viel Zeit hast du täglich?" (5 Min / 10 Min / 15 Min+)

Screen 8: Therapie-Hinweis (EINMALIG, freundlich)

- "Diese App ersetzt keine Therapie. Wenn du merkst dass du mehr brauchst, bist du nicht schwach – du bist mutig. Unter 'Hilfe finden' zeigen wir dir wie du den ersten Schritt machst."
- Bestätigen mit: "Verstanden, ich starte"

Screen 8b: Modus-Auswahl (NEU)

- "Wie möchtest du die App nutzen?"
- Zwei Optionen: Mit Spiel-Elementen / Klartext
- "Du kannst das jederzeit in den Einstellungen ändern"

Screen 9: Erster Pfad wird aufgebaut

- Animation: Profil wird erstellt (Klartext: schlichte Animation)

- "Deine erste Übung wartet auf dich"
- Direkt zur ersten Übung (max. 3 Minuten)
- ERSTES ERFOLGSERLEBNIS IN MINUTE 3

Community-Feature

Prinzip: Stille Solidarität – kein Vergleich, kein Druck

```
interface CommunityEntry {
    id: string
    text: string           // Anonym, max 100 Zeichen
    confirmCount: number   // Andere haben "Ich kenne das" gedrückt
    createdAt: Date
    // KEINE Namen, KEINE Fotos, KEINE Likes
}
```

Was angezeigt wird:

- Anonyme Aussagen anderer ("Heute war schwer, aber ich war da")
- Ein stiller **"Ich kenne das"** Button – kein Kommentar möglich
- Anzahl aktiver Nutzer heute: *"X Menschen haben heute ihre App geöffnet"*
- **KEINE** Ranglisten, **KEINE** Vergleiche

Therapie-Hinweis System (`TherapyHint`)

Der Hinweis erscheint **nie aufdringlich**, aber **immer wenn nötig**:

```
function shouldShowTherapyHint(context: AppContext): boolean {
    return (
        context.screen === 'onboarding' || // Einmalig
        context.topic?.isSensitive === true || // Trauma,
        context.userStreak === 0 && context.daysSinceInstall > 14 || // Lange Pa
        context.userRequested === true // Nutzer ö
    )
}
```

Ton: Einladend, nicht beängstigend. *"Du bist mutig"* statt *"Du brauchst Hilfe."*

Krisentelefon ist auf dem `HelpScreen` IMMER sichtbar:

- Telefonseelsorge: **0800 111 0 111** (kostenlos, 24/7)
-

Design-System

```
const COLORS = {
  // Hauptfarben
  primary:    '#0D9488',    // Teal - Ruhe, Vertrauen
  dark:       '#0F1F2E',    // Fast-Schwarz - Tiefe, Sicherheit
  background: '#F1F5F9',    // Hellgrau - sauber, nicht klinisch

  // Akzente
  amber:      '#D97706',     // Energie, Aufmerksamkeit
  purple:     '#7C3AED',     // Tiefe Skills, Progression
  rose:       '#E11D48',     // Wichtige Hinweise
  blue:       '#1D4ED8',     // Reflexion, Ruhe

  // Semantisch
  success:    '#10B981',
  warning:    '#F59E0B',
  danger:     '#EF4444',
  textSoft:   '#475569',

  // Akut-Hilfe (NEU)
  helpNow:    '#0D9488',     // Ruhiger Teal-Ton, nicht alarmierend
}

const TYPOGRAPHY = {
  fontFamily: 'Nunito',      // Freundlich, lesbar, nicht klinisch
  // Sizes: 12 / 14 / 16 / 18 / 22 / 28 / 36
}

// Sprache: Kumpelhaft, direkt, nie von oben herab
// Klartext-Modus: sachlich, respektvoll, ohne Metaphern
// Fehler-Meldungen: Nie "Das hast du falsch gemacht" → "Kein Problem, weiter geh"
```

Design des Akut-Buttons:

- Sichtbar aber nicht rot/alarmierend – ein ruhiger Teal-Ton
- Beschriftung "Jetzt helfen" (nicht "Notfall", nicht "Krise")
- Position: rechts oben in der Navigation, auf jedem Screen sichtbar
- Haptisches Feedback beim Tap, sofortige Reaktion

Datenschutz & Ethik

- **Alle Daten lokal** (AsyncStorage / SQLite) – kein Cloud-Zwang
 - **Kein Account nötig** – optionaler Account nur für Geräte-Backup
 - **Kein Tracking**, keine Werbung, kein Verkauf von Daten
 - **Berichte nur auf expliziten Wunsch** – niemals automatisch geteilt
 - **Keine Diagnosen** – die App hilft zu verstehen, nicht zu etikettieren
 - **Krisenprotokoll:** Krisentelefon ist immer im `HelpScreen` erreichbar, der Akut-Button immer im Header
-

MVP – Was zuerst gebaut wird

Phase 1 (MVP):

- Onboarding (Screens 1–9 inkl. Modus-Auswahl 8b)
- HomeScreen mit Tages-Übung
- 10 erste Übungen (Text + Timer, noch ohne Audio)
- 4 erste Skills (Gedanken beobachten, Innere Ruhe, Grounding, Innerer Anker)
- **Akut-Hilfe-Button mit 6 Zuständen und 8–10 Akut-Übungen**
- **Klartext-Modus (vollständig)**
- Einfaches Charakter-System (Level + XP, noch ohne Equipment-Sprites)
- Wochenplan (einfach, ohne Verknüpfung zu Übungen)
- Zielsystem (Großes Ziel + Schritte)
- TherapyHint + HelpScreen (statische Infoseiten)

Phase 2:

- Audio-Übungen (Expo AV)
- Animationen (Lottie – Atemvisualisierung)
- Spaced Repetition Algorithmus

- Reflexions-Journal + Nachricht ans zukünftige Ich
- Vollständige Themen-Bibliothek (14 Themen)
- Charakter Equipment & Sprites
- PDF-Export Fortschrittsbericht
- Erweiterte Akut-Übungen (mit Audio-Guides)

Phase 3:

- Community-Feature (anonyme stille Solidarität)
- Therapie-Orientierungshelfer (5-Fragen-Flow)
- Push-Notifications (sanft, opt-in)
- Alle 7 Skills mit Level 1–5
- Lernendes Akut-Matching (verbesserte Vorschläge basierend auf Nutzungshistorie)



Nächste Schritte – Content & Konzept

Diese Arbeiten können ohne Entwicklungsumgebung erledigt werden – reine Konzept- und Textarbeit. Priorisiert nach Wichtigkeit für den MVP.



Priorität 1 – Akut-Übungen ausschreiben

Die 6 Zustände stehen, die Übungs-IDs sind noch Platzhalter. Für jede Akut-Übung werden benötigt:

- **Bildschirm-Text:** Exakt die Worte, die der Nutzer in jedem Schritt liest
- **Dauer:** Sekundengenau
- **Was die App anzeigt:** Timer, Animation, Atemrhythmus etc.
- **Abschluss-Satz:** Der letzte Satz nach der Übung

Reihenfolge: Mit *“Alles ist zu viel”* starten (universal, häufigster Zustand). Erste Übung: **5-4-3-2-1 Grounding** – alle 5 Schritte ausschreiben.



Höchste Priorität: Wer um 2 Uhr morgens den Button drückt, liest genau diese Worte. Jedes Wort zählt.

✅ Priorität 2 – 10 MVP-Übungen textlich fertigstellen

4 Skills im MVP, je 2–3 Übungen auf Level 1:

Skill	Methode	Übungen (Level 1)
Gedanken beobachten	CBT	Gedanken benennen, Wolken-Metapher
Innere Ruhe	MBSR	Atemraum, Body Scan (kurz)
Grounding	DBT	5-4-3-2-1, Füße spüren
Innerer Anker	ACT	Werte-Anker, sicherer Ort

Pro Übung: Titel, Anleitung (Schritt-für-Schritt), Dauer, Reflexionsfrage.

✅ Priorität 3 – Onboarding-Texte final schreiben

Alle 9+1 Screens haben Struktur, aber noch keine finalen Texte. Besonders wichtig:

- **Screen 1** – setzt den Ton für die gesamte App
 - **Screen 8** – Therapie-Hinweis: 3 Varianten schreiben, laut vorlesen, beste auswählen
 - **Screen 8b** – Modus-Auswahl: Beschreibung beider Modi muss in 2 Sätzen klar sein
-

✅ Priorität 4 – Reflexionsfragen sammeln

Nach jeder Übung kommt eine Reflexionsfrage. Ziel: 30–40 Fragen, übungsgebunden, nicht generisch.

Schlecht: *“Wie fühlst du dich?”* **Gut:** *“Was hat sich in deinem Körper verändert, während du gezählt hast?”*

Sammlung in einer Notiz, später den Übungen zuordnen.

✅ Priorität 5 – Konkurrenz-Analyse (Handy)

Apps zum Durchlaufen und vergleichen:

- **7Mind** – Achtsamkeit, deutsch
- **Selfapy** – CBT-basiert, deutsch, für Depression/Angst

- **MindDoc** – Stimmungstracking, deutsch
- **Woebot** – Chatbot-basiert, CBT
- **Headspace / Calm** – als Negativbeispiel für Mental Buddy's Zielgruppe

Für jede App notieren:

1. Wo ist die Hürde am höchsten für einen Mann mit Depression?
2. Was fühlt sich von Sekunde 1 falsch an?
3. Was macht Mental Buddy konkret anders?

✅ **Priorität 6 – Themen-Bibliothek (4 MVP-Themen)**

14 Themen gesamt, für den MVP die 4 häufigsten zuerst:

1. Angst & Panik
2. Antriebslosigkeit & Depression
3. Innerer Kritiker & Selbstzweifel
4. Wenn alles zu viel wird (Overwhelm)

Pro Thema: `introText` (sanft, Woche 1–2) + `deepText` (klar, ab Woche 3) + `therapyNote` .

🕒 **Wartet auf PC / Entwicklungsumgebung**

- Expo-Projekt initialisieren
- Navigation aufbauen (Bottom Tabs)
- `settingsStore` (Zustand) implementieren
- `ExercisePlayer` Komponente
- SQLite Schema
- Akut-Matching Logik (`acuteMatcher.ts`)

💡 **Kern-Prinzipien für die Entwicklung**

1. **Kein Zwang, niemals** – Keine roten Zahlen, keine Strafen, kein Shame

2. **Kleine Schritte** – Jede Übung max. 5 Minuten. Lieber 3 Min täglich als 30 Min nie
3. **Viele Erfolgserlebnisse** – Level-Ups, XP, abgeschlossene Übungen sollen sich gut anfühlen (im Klartext-Modus: ruhige Bestätigung statt Feuerwerk)
4. **Transparenz** – Jeder Skill zeigt welche Methode dahinter steckt
5. **Zurückgehen ist erlaubt** – Niedrigeres Level wählen ist eine Stärke, kein Versagen
6. **Therapie betonen ohne zu erschrecken** – Einladend, mutig machend, nie stigmatisierend
7. **Datenschutz first** – Nichts verlässt das Gerät ohne explizite Zustimmung
8. **Hilfe ist immer einen Tap entfernt** – Der Akut-Button ist nie versteckt, nie verschachtelt
9. **Der Nutzer wählt seine Sprache** – RPG oder Klartext. Beides ist richtig.